Operating system II

Part III

Inter-Process Communication (I P C)

K.Belarbi

Université des sciences et de technologie Med Boudiaf Oran
Faculté MI Département Informatique
10 Avril 2025

20 avril 2025

Inter-Process Communication

Introduction

Les process concurrents s'exécutant dans OS peuvent être :

- **Indepedent process** : si le process n'affectent pas d'autres process en cours d'exécution ou ne peut pas être affecté par eux ;
  An *independent process* est un process qui ne partagent pas de data (for instance or a shared file) avec d'autres process.
- **Cooperatif process** : si le process affectent d'autres process en cours d'exécution ou peut être affecté par eux ;
  Tous process qui partagent des data avec d'autres process est un *a Cooperating process*.

Inter-Process Communication

Cooperatif process

Pourquoi proposer un environnement permettant la coopération des process ?

- **Information sharing :** fournir un environnement permettant l'accès concurrents dans le cas ou plusieurs utilisateurs peuvent être intéressés par la même information (exp. un fichier partagé) ;
- **computation speedup :** découper les tâches en sous tâches, chacune d'elles s'exécutant en parallèle avec les autres ;
- **Modularity :** construire un système de manière modulaire, en divisant les fonctions du système en process séparés ;
- **Convenience :** Un utilisateur peut travaillé plusieurs tâches en même temps. (Exemple : éditer, imprimer et compiler en même temps).

Inter-Process Communication (IPC)

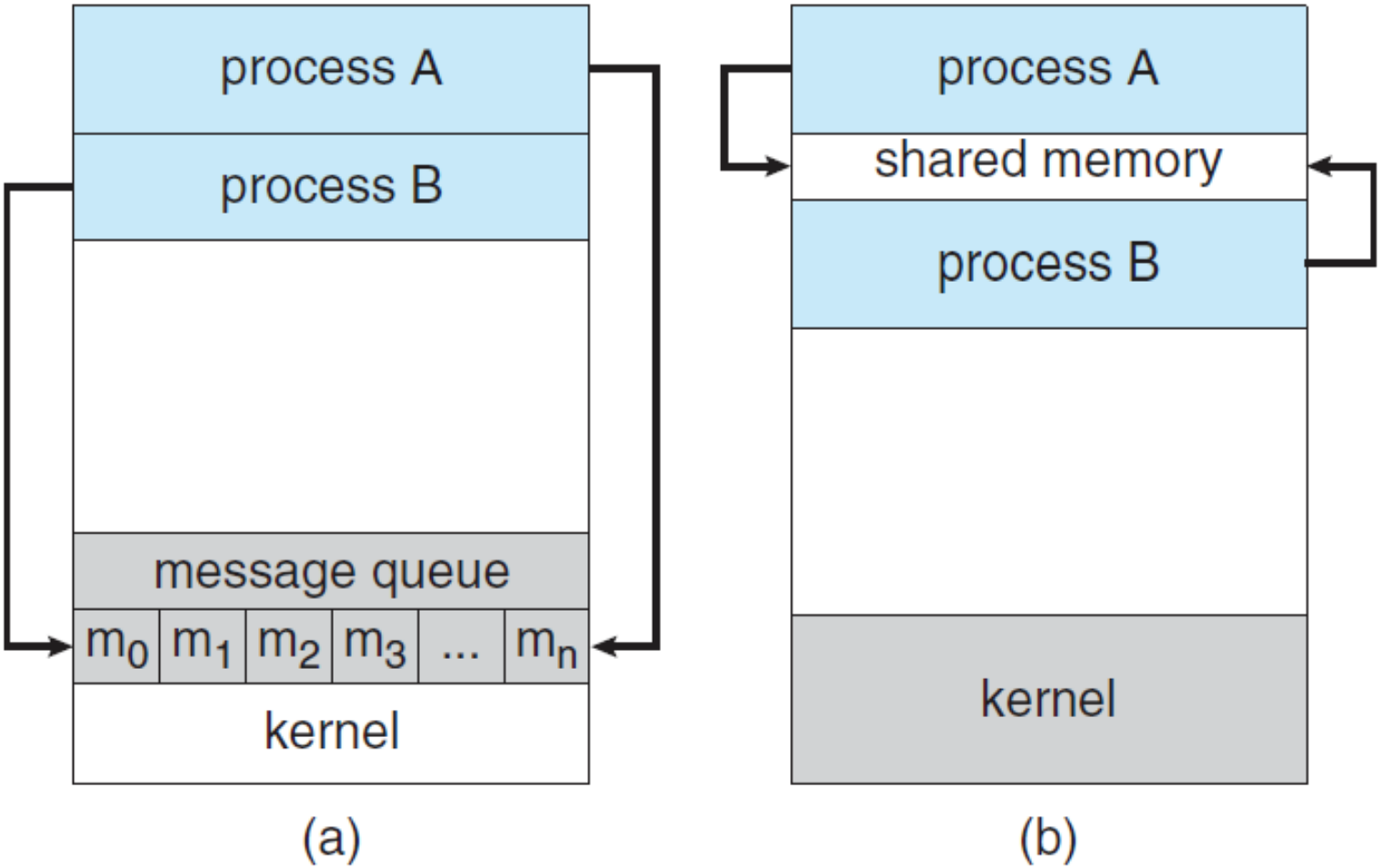
InterProcess Communication

L'exécution concurrentiels, qui demande la coopération entre process requiert des mécanismes permettant aux process de **communiquer entre eux** Interprocess communications (IPC) et de synchroniser leurs actions.

- OS doit fournir aux cooperating process le moyen de communiquer entre eux en utilisant une fonction interprocess communication (IPC) ;
- IPC fournit un mécanisme permettent aux process de communiquer et de synchroniser leurs actions ;
- Les processus peuvent communiquer entre eux via :
  - i) Shared memory ;
  - ii) Message passing

Inter-Process Communication (IPC)

Figure – Communications models. (a) Message passing. (b) Shared memory



Inter-Process Communication (IPC)

Deux modèles de communication

- **Message passing** *(Données échangées entre processus)*
  - Chacun les traite à sa façon
  - Un seul processus a les données à la fois
- **shared memory** *(Données partagées entre processus)*
  - Données communes : une région de mémoire partagée réside dans l'espace d'adressage du processus qui crée la région de mémoire partagée.
  - Accès et modifications non exclusives
- IPC est mieux fournie par un système de message (définit de plusieurs manières différents) ;
- Les schémas de communications de la mémoire partagée et du système de messages ne sont pas exclusifs et peuvent être utilisés simultanément dans un OS unique et même dans un process unique.

- Les process partagent un ensemble de données communes qui sont soit en mémoire *(variables ou segments de données)* soit sur disque (*fichiers*) ;
- Chaque process peut accéder en lecture ou en écriture à cet ensemble de données appelé **espace de données commun** ;
- Des problèmes peuvent se poser lorsque plusieurs process essayent d'accéder en même temps à un espace commun.

the producer– consumer problem

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
}item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

```
item next_produced;

while (true) {
     /* produce an item in next_produced */

     while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
}
```

```
item next_consumed;

while (true) {
     while (in == out)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;

     /* consume the item in next_consumed
}
```

Deux moyens de communication interprocess offerts par le système Unix/Linux : **les signaux** et les envois de messages par le biais de **tubes**.

- **Les tubes** :
  - **Les tubes anonymes** (ou **pipes**), créés avec la fonction pipe() dans C, pour les processus qui on un ancêtre commun, typiquement, un parent et son enfant après fork().
    - Ils ne possèdent pas de nom de fichier et ne sont donc pas visible dans le système de fichier.
    - Les tubes anonymes sont par default unidirectionnel.
  - **Les tubes nommés** (ou **fifos**), créés avec mkfifo(), et qui ont une entrée dans un répertoire, pour les processus qui ne se connaissent pas.
    - Ils sont stockés sous forme de fichiers spéciaux dans le système de fichier et sont accessible via un chemin.
- Les **interruptions logicielles** ou **signaux** : l'envoi de signaux avec kill(), raise(), alarm() et le traitement a leur leur réception par un **handler** installe préalablement par signal() ou sigaction().

Dans un shell Linux, le symbole | crée un tube.

Example1

$ ls | less

provoque la création par le shell de deux processus fils, l'un pour *ls* et l'autre pour *less*

Le shell crée également un tube connectant **la sortie standard** du processus ls avec **l'entrée standard de less. Les noms des fichiers listés par ls sont envoyés à less dans le même ordre que s'ils étaient envoyés directement au terminal.**

Example2

$ cat fichier1 fichier2 fichier3 | sort > /dev/lp

demande à la commande cat de concaténer les trois fichiers (fichier1, fichier2 et fichier3) et d'envoyer le résultat de l'opération à la commande sort qui triera les lignes dans l'ordre alphabétique. La sortie du tri sera alors envoyée sur l'imprimante.

$ cat fichier1 fichier2 | sort > /dev/lp &

Si l'utilisateur termine sa commande par un " & ", le shell n'attend pas la fin de la commande ; il renvoie immédiatement le prompt. il démarre le tri comme un job en tâche de fond, ce qui permet à l'utilisateur de continuer à travailler normalement pendant que le tri s'effectue.

Le tube anonyme : création avec pipe()

- Le tube anonyme ou pipe est un mécanisme de communication unidirectionnel entre processus qui ont un ancêtre commun via fork().
- Un pipe est créé via la fonction C eponyme pipe() : Elle admet comme paramètre **un tableau de deux entiers**.

```
# include <unsitd.h>
int pipe (int fds[2]) ; // returns 0 if OK,-1 on error
int fr ;
int fw ;
pipe (pipe_fds) ;
fr = pipe_fds[0] ;
fw = pipe_fds[1] ;
```

- Deux descripteurs sont ouverts sur le même flot de données (une file de caractères de capacité fixe) qui transite via le kernel, sur lesquels on utilise read(), write(), close().
  - pipe_fds[0] est ouvert en lecture sur le pipe,
  - pipe_fds[1] est ouvert en écriture sur le pipe.

- Opérations non autorisées sur les tubes anonyme

Anonymes signifie sans référence
$\Longrightarrow$ Impossible d'appliquer des primitives nécessitant le nommage (ex. : open, rename, chmod, ...)

- Lecture dans un tube

Primitive read

```
nb_lu = read(tube[0], buf, TAILLE_BUF);
```

où
tube[0] : *descripteur en lecture*
buf : *pointeur sur TAILLE_BUF*
nb_lu : *nombre de caractères lus*

- Comment ça marche ?

```
si      le tube ≠ 0 et contient taille caractères
        alors     la primitive extrait du tube min(taille, TAILLE_BUF)
                  caractères et les places à l'adresse buf
sinon
        si    nbre écrivains = 0, fin de fichier atteinte
              alors    pas de caractère lu et renvoie nb_lu=0 par primitive
        fsi
        si    nbre écrivains ≠ 0 alors
              si lecture bloquante alors processus mis en sommeil jusqu'à
                 tube non vide ou 0 écrivain
              fsi
        si lecture non bloquante alors retour immédiat
              et valeur retour = -1 et errno = EAGAIN
        fsi
fsi
```

- Écriture dans un tube

Primitive write

nbEcrit = write(tube[1], buffer, taillebuf) ;

où
tube[1] : *descripteur en écriture*
buffer : *pointeur sur taillebuf*
taillebuf : *caractères accessibles*

Rq : Si le nombre de caractères à écrire < à la taille du tube l'écriture sera atomique sinon décomposition arbitraire

- **Comment ça marche ?**

```
Si    nbre lecteurs dans tube = 0
      alors     SIGPIPE délivré au processus écrivain ⇒ terminaison processus
sinon
      si écriture bloquante
      alors       retour primitive vrai que si les n
                  caractères sont écrits et processus endormi jusqu'à tube vide ;
      sinon
            si n > PIPE_BUF alors le retour est un nombre< n ; fsi ;
            si n ⩽ PIPE_BUF et nbre emplacements libres ≥ n
               alors     écriture atomique est réalisé ;
            fsi ;
            si n ⩽ PIPE_BUF et nbre emplacements libres < n
               alors retour immédiat sans écriture ;
               (renvoi –1 et errno = EAGAIN) ;
            fsi ;

FSI ;
```

- **Exercice1 :** Écrire dans un tube sans lecteur.
- **Exercice2 :** Utiliser un fichier pour faire communiquer un processus et son père : le fils exécute "ls –l" et le père "wc –l"
- **exercice3 :** Le programme traduit la tentative de lecture d'un processus alors qu'il n'y a pas de rédacteur

Le tube nommés fifo First_In First_Out : création avec mkfifo()

- Un tube nommé est un mécanisme de communication unidirectionnel entre processus qui a une entrée dans un répertoire comme un fichier classique.
- Permettent aux processus la communication en mode flot
- Identifiables via la commande shell ls –l
- On crée un fifo soit avec la commande mkfifo :

```
$ mkfifo -m 0666 somefifo      # 0666 is octal for mode u=rw,g=rw,o=rw
$ ls -l somefifo
prw-rw-rw- 1 regis prof 0 avril  5 14:54 somefifo
$ rm fifoname
```

.... ou par la fonction C eponyme mkfifo() :

```
#include <stdlib.h>
#include <stdio.h>
```

```
#include <sys/types.h>
#include <sys/stat.h>
```

```
// mkfifo.c
int main (void) {
  umask (0);
  if (mkfifo ("somefifo", 0600) == -1) { perror ("mkfifo"); exit (1); }
  fprintf (stderr, "created fifo 'somefifo'\n");
  return 0;
}
```

```
$ ./mkfifo
created fifo 'somefifo'
$ ls -l somefifo
prw------- 1 regis prof 0 avril  5 14:54 somefifo
```

```
$ ./mkfifo
mkfifo: File exists
$ rm somefifo
```

Exemple : le modèle producteur-consommateur avec un tube nommé

mkfifo()

```
#include <sys/types.h>
#include <sys/stat.h>

int mkfifo(const char *nomfichier, mode_t mode);
```

"*nomfichier*" définit le chemin d'accès au tube nommé ;
"*mode*" les droits d'accès des différents utilisateurs à cet objet.
Un fifo s'ouvre avec open() comme un fichier régulier. Par défaut, l'ouverture est bloquante en lecture et en écriture :

ouverture avec open

```
/* processus 1 */

int d_lecture, d_ecriture;
...
d_ecriture=open("fifo1", O_WRONLY);
d_lecture=open("fifo2", O_RDONLY);

/* process 2 */

int d_lecture, d_ecriture;
...
d_ecriture=open("fifo2", O_WRONLY);
d_lecture=open("fifo1", O_RDONLY);
```

Le flag O_NONBLOCK rend l'ouverture et les IO non-bloquantes :

- `int fd= open ("somefifo", O_RDONLY | O_NONBLOCK);`

l'ouverture en lecture retourne immédiatement même s'il n'y a pas d'écrivain.

- `int fd= open ("somefifo", O_WRONLY | O_NONBLOCK);`

l'ouverture en écriture retourne aussi immédiatement, mais échoue avec fd = -1 si le fifo n'a pas de lecteur.

InterProcess Communication

Les Sockets

- **Un socket** est un dispositif de communication bidirectionnel pouvant être utilisé pour communiquer avec un autre processus sur la même machine ou avec un processus s'exécutant sur d'autres machines.
- Communication distante : faire communiquer des applications sur des ordinateurs distincts
  - Peut aussi être utilisé pour des applications sur une même machine
  - Majoritairement pour du clients-serveur
- Les programmes Internet comme Telnet, rlogin, FTP, talk et le World Wide Web utilisent des sockets.
- Pour créer un socket, il faut indiquer trois paramètres : le style de communication, l'espace de nommage et le protocole.

InterProcess Communication

Les Sockets

```
#
include <sys/types.h>
#include <sys/socket.h>
int socketpair ( int domain, int type, int
protocol, int sv[2]) ;
```

Crée 2 sockets associées.
Pour le faire via une communication locale :

- domain = AF_UNIX
- protocol = 0
- type :
  - SOCK_STREAM : communication par flot (comme pour les tubes)
  - SOCK_DGRAM : communication en mode paquet
- sv : un tableau de 2 entiers, pour le renvoi des 2 descripteurs de fichiers (les 2 sockets)

InterProcess Communication

Les Sockets

mode paquet (DGRAM)

Au contraire du mode flot (STREAM), chaque information envoyée constitue une entité indivisible.

Exemple :

- write(in,"ABC",3)
  - la file de messages contient "ABC"
- write(in,"123",3)
  - la file contient "ABC","123"
- read(out,bf,10)
- renvoie 3, et bf contient "ABC"
  - la file contient "123"
- read(out,bf,10)
- renvoie 3, et bf contient "123"
  - la file vide
- read(out,bf,4)
  - bloque jusqu'à ce qu'un processus écrive dans le socket...

- Les **interruptions logicielles** où les **signaux** sont utilisées par l'OS pour aviser les processus utilisateurs de l'occurrence d'un événement important ;
- les signaux sont utilisés pour établir une communication minimale entre process, une communication avec le monde extérieur et faire la gestion des erreurs. Ainsi, un process peut :
  - Ignorer le signal.
  - Appeler une routine de traitement fournie par le noyau. Cette procédure provoque normalement la mort du processus.
  - Appeler une procédure spécifique créée par le programmeur. Tous les signaux ne permettent pas ce type d'action.

Tous les signaux ont une routine de service, ou une action par défaut. Cette action peut être du type

- A : terminaison du processus
- B : ignorer le signal
- C : Créer un fichier core
- D : Stopper le processus
- E : La procédure de service ne peut pas être modifiée
- F : Le signal ne peut pas être ignoré

Chaque signal est identifié au moyen d'un numéro entier positif.

Signal	Num	Type	Description
SIGKILL	9	AEF	Termine l'exécution du process
SIGSTOP	19	DEF	Stoppe le process
SIGCONT	18	•	Continue l'exécution du process
SIGCHILD	17	B	Un process fils est fini
SIGUSR1	10	A	Signal d'utilisateur
SIGUSR2	10	A	Signal d'utilisateur

Envoi de signal avec kill(), raise(), et alarm()

- kill() est la fonction C éponyme de la commande kill.
- kill() envoie le signal signum au processus pid.
- Lorsque signum = 0, aucun signal n'est envoyé, mais le test d'erreur permet de vérifier l'existence du processus.
- Si le processus appelant n'a qu'un seul thread d'exécution, alors raise (signum) est un wrapper sur kill (getpid(), signum).
- La fonction envoie le signal signum au processus appelant.
- alarm(n) fait en sorte que SIGALRM sera envoyé dans n secondes.
- Une nouvelle alarme remplace l'éventuelle alarme en cours.
- Le nombres de secondes restant pour cette dernière est retourné.
- alarm(0) annule toute disposition d'alarme en cours.

Exemple 1. Le programme montre l'utilisation du masquage des signaux.

```
1 # include < stdio.h>
2 # include < signal.h>
3 # include < unistd.h>
4
5 void sighandler ( int signum) ;
6 int main( void )
7 {
8   char buffer[256] ;
9   if ( signal(SIGTERM,&sighandler ) = = SIG_ERR )
10  {
11     printf ( "Ne peut pas manipuler l e s igna l \n" ) ;
12     exit ( 1 ) ;
13  }
14 while ( 1 )
15  {
16     fgets ( buffer , sizeof ( buffer ) , stdin ) ;
17     printf ( " Input : %s " , buffer ) ;
18  }
19 return 0 ;
20 }
21
22 void sighandler(int signum)
23 {
24 printf("Masquage du signal SIGTERM\n");
25 }
26
```

Exemple 2. Le programme illustre l'utilisation des signaux SIGUSR1 et SIGUSR2

```
1 # include < ios t ream . h>
2 # include < uni s td . h>
3 # include <wait . h>
4 static void action ( int sig ) ;
5 void main ( )
6 {
7    int i , pid , etat ;
8    // Spe c i f i c a t i o n de l ' a c t ion du s igna l
9
10   if ( signal ( SIGUSR1 , action ) = = SIG_ERR )
11      cout<<" Er reur de t r a i t ement du code de l ' a c t ion " << endl ;
12   if ( signal ( SIGUSR2 , action ) = = SIG_ERR )
13      cout<<" Er reur de t r a i t ement du code de l ' a c t ion " << endl ;
14   if ( ( pid = fork ( ) ) = = 0 )
15      {
16        k i l l ( getppid ( ) , SIGUSR1 ) ; // Envoyer s igna l au parent .
17        for ( ; ; )
18        pause ( ) ; // Mise en a t t e n t e d' un s igna l
19       }
20    else
21       {
22        kill ( pid , SIGUSR2) ; // Envoyer un s igna l a l ' enfant
23        cout<<" Parent : terminai son du f i l s " << endl ;
24        kill ( pid , SIGTERM) ; // Signal de terminai son a l ' enfant
25        wait (&etat ) ; // attendr e l a f i n de l ' enfant
26        cout << " Parent : f i l s termine " << endl ;
27        }
28
29 // Fonct ion de f i n i s s a n t l e code de
30 // l ' a c t ion a l a r e c ept ion d' un s igna l
31
32 static void action ( int sig )
33 {
34    switch ( sig )
35      {
36        case SIGUSR1 :
37        cout << " Parent : s igna l SIGUSR1 recu " << endl ;
38        break ;
39        case SIGUSR2 :
40        cout << " F i l s : s igna l SIGUSR2 recu " << endl ;
41        break ;
42        defaul t : break ;
43      }
44 }
45
46
```

InterProcess Communication

Les messages (Structure de base)

La fonction d'un système de message est de permettre aux process de communiquer entre eux sans avoir recoure aux variables ;
IPC propose deux fonctions :

- send(*message*)
- receive(*message*)

Communication directe :
- **send**(*P*,*message*) : Envoie un message au process P ;
- **receive**(*Q*,*message*) : Reçoit un message du process Q.

Propriétés de cette liaison

- établir automatiquement la liaison entre chaque pair de process. Ils doivent seulement connaître l'identité des autres pour communiquer ;
- une liaison est associée à exactement deux process ;
- entre chaque pair de process, il existe une seule liailson ;
- la liaison peut être unidirectionnelle, mais généralement elle est bidirectionnelle.

InterProcess Communication

Les messages (Structure de base)

Solution pour le problème Producteur/Consommateur :

Process Producteur

repeat ;

...

produire un élément dans nextp

...

send(consommateur,nextp)

until *false* ;

Process consommateur

repeat ;

receive(producteur,nextp)

...

consommer l'élément de nextp

...

until *false* ;

InterProcess Communication

Les messages (Structure de base)

La communication indirecte

- les messages sont envoyés et reçus à une boites aux lettres (ports)
- une boite aux lettres est vu comme un objet ou un process peut placer des messages et peuvent être supprimer ;
- chaque boîte aux lettre possède un identificateur unique ;
- deux process ne peuvent communiquer que s'ils partagent une boite aux lettres ;
- les primitives send et receive sont définis comme suit :
  - send(A, messsage) envoie un message à la boite aux lettres A ;
  - receive (A, messsage) reçoit un message à la boite aux lettres A ;
- une boite aux lettre peut appartenir au process où au un système ;
- une boite aux lettre appartenant à OS existe par elle-même et est indépendante ;

InterProcess Communication

Les messages (Structure de base)

Bufferisation

- une liaison possède une capacité pour la quantité de message pouvant y résider temporairement ;
- cette propriété peut être comme une file d'attente de message attaché à la liaison ;
- deux manière d'implémenter une file d'attente :
  - **Capacité zéro** ⇒ long(liste d'attente)==0 ⇒ aucun message en attente ⇒ synchronisation appelée **rendez-vous**
  - **capacité limitée** ⇒ long(liste d'attente==n ⇒

InterProcess Communication

Les messages (Structure de base)

Messages perdus

- un message peut se perdre dans le réseau de communication à cause d'une panne matérielle ou dans la ligne de communication. Trois méthodes pour en faire face :
  1. OS a la responsabilité de détecter cet événement et de renvoyer le message ;
  2. le process émetteur a la responsabilité de détecter cet événement et de retransmettre le message, s'il le désire ;
  3. OS a la responsabilité de détecter cet événement ; il notifie aux process émetteur qu'un message est perdu. Le process émetteur peut poursuivre comme il le désire ;
- Utilisation des temporisateurs pour détecter qu'un message est perdu.

InterProcess Communication

Les messages (Structure de base)

Messages embrouillés

- un message peut être remis à destination, seulement il peut être embrouillé à cause du bruit dans le canal ;
- l'OS retransmet le message d'origine ;
- Pour détecter ce type d'erreur, on utilise les sommes de contrôle CRC ;

InterProcess Communication

Example : POSIX Shared Memory

Figure – Consumer process illustrating POSIX shared-memory API.

```
#include <stdio.h>
#include <stlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

InterProcess Communication

Example : POSIX Shared Memory

Figure – Producer process illustrating POSIX shared-memory API.

```
#include <stdio.h>
#include <stlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

   /* create the shared memory object */
   shm_fd = shm_open(name, O_CREAT | O_RDRW, 0666);

   /* configure the size of the shared memory object */
   ftruncate(shm_fd, SIZE);

   /* memory map the shared memory object */
   ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

   /* write to the shared memory object */
   sprintf(ptr,"%s",message_0);
   ptr += strlen(message_0);
   sprintf(ptr,"%s",message_1);
   ptr += strlen(message_1);

   return 0;
}
```